

```

1 #=====
2 # MIT 6.046J 代码精读 • PS7 - Dinic/Edmonds-Karp 最大流
3 # 源文件: maxflow.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS7: a real max-flow solver (Dinic + Edmonds-Karp) with the applications
7 needed by Problems 7-1..7-3.
8
9 Graph is stored as an adjacency list of directed residual edges; each edge has a
10 `cap`, current `flow`, and a pointer to its reverse edge. Integer capacities.
11 """
12 from __future__ import annotations
13
14 from collections import deque
15 from typing import Dict, List, Optional, Tuple
16
17
18 class Edge:
19     __slots__ = ("to", "cap", "flow", "rev")
20
21     def __init__(self, to: int, cap: int, rev: int):
22         self.to = to
23         self.cap = cap
24         self.flow = 0
25         self.rev = rev # index of the reverse edge in graph[to]
26
27     def residual(self) -> int:
28         return self.cap - self.flow
29
30
31 class MaxFlow:
32     def __init__(self, n: int):
33         self.n = n
34         self.graph: List[List[Edge]] = [[] for _ in range(n)]
35
36     def add_edge(self, u: int, v: int, cap: int) -> Tuple[int, int]:
37         """Add directed edge u->v with capacity cap (and a 0-cap reverse).
38         Returns (u, index) so the edge can be located later (for dynamic updates)."""
39         self.graph[u].append(Edge(v, cap, len(self.graph[v])))
40         self.graph[v].append(Edge(u, 0, len(self.graph[u]) - 1))
41         return (u, len(self.graph[u]) - 1)
42
43     # ----- Dinic -----
44     def _bfs_level(self, s: int, t: int) -> Optional[List[int]]:
45         level = [-1] * self.n
46         level[s] = 0
47         q = deque([s])
48         while q:
49             u = q.popleft()
50             for e in self.graph[u]:
51                 if e.residual() > 0 and level[e.to] < 0:
52                     level[e.to] = level[u] + 1
53                     q.append(e.to)
54         return level if level[t] >= 0 else None
55
56     def _dfs_blocking(self, u: int, t: int, pushed: int, level: int) -> int:
57         if u == t:
58             return pushed
59         while it[u] < len(self.graph[u]):
60             e = self.graph[u][it[u]]
61             if e.residual() > 0 and level[e.to] == level[u] + 1:
62                 d = self._dfs_blocking(e.to, t, min(pushed, e.residual()), level, it)
63                 if d > 0:
64                     e.flow += d
65                     self.graph[e.to][e.rev].flow -= d
66                     return d
67             it[u] += 1
68         return 0
69
70     def max_flow(self, s: int, t: int) -> int:
71         flow = 0
72         while True:
73             level = self._bfs_level(s, t)
74             if level is None:
75                 break
76             it = [0] * self.n
77             while True:
78                 f = self._dfs_blocking(s, t, float("inf"), level, it)
79                 if f == 0:
80                     break
81             flow += f
82         return flow
83
84     def value(self, s: int) -> int:
85         return sum(e.flow for e in self.graph[s] if e.cap > 0)

```

```

86
87 # ----- utilities for applications -----
88 def min_cut_reachable(self, s: int) -> List[bool]:
89     """Vertices reachable from s in the residual graph (the s-side of a min cut)."""
90     seen = [False] * self.n
91     seen[s] = True
92     q = deque([s])
93     while q:
94         u = q.popleft()
95         for e in self.graph[u]:
96             if e.residual() > 0 and not seen[e.to]:
97                 seen[e.to] = True
98                 q.append(e.to)
99     return seen
100
101 def extract_paths(self, s: int, t: int, k: int) -> List[List[int]]:
102     """Decompose the current s-t flow into k edge-disjoint paths (each edge
103     carrying one unit; assumes unit capacities used for disjoint-paths)."""
104     # follow used edges greedily
105     used = [[False] * len(self.graph[u]) for u in range(self.n)]
106     paths = []
107     for _ in range(k):
108         path = [s]
109         u = s
110         ok = True
111         while u != t:
112             nxt = None
113             for idx, e in enumerate(self.graph[u]):
114                 if e.cap > 0 and e.flow > 0 and not used[u][idx]:
115                     nxt = (idx, e)
116                     break
117             if nxt is None:
118                 ok = False
119                 break
120             idx, e = nxt
121             used[u][idx] = True
122             e.flow -= 1 # consume this unit for extraction
123             u = e.to
124             path.append(u)
125         if ok:
126             paths.append(path)
127     return paths
128

```